

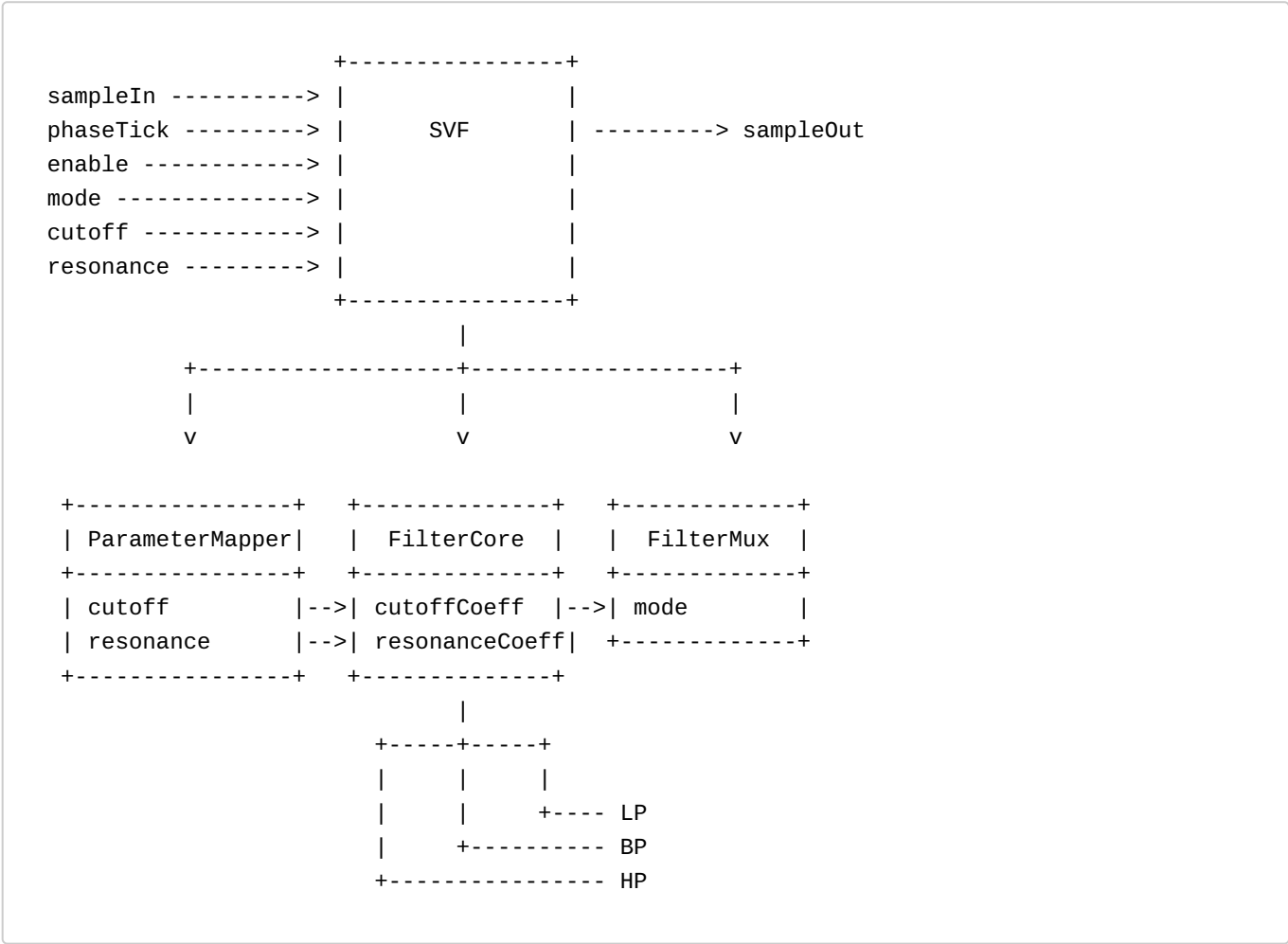
spinalSynth: Filter Specification

Description

The Filter Module processes audio samples within the spinalSynth signal path.

The module architecture shall allow future filter core implementations without changing the external interface.

Architecture



Timing

Main system clock

Signal	Frequency
clk	24 MHz

External: Sample Interface Sync

The Filter Module receives and transmits audio samples at a rate of 480 kHz. Input and output samples are transferred using SpinalHDL Flow interfaces. A dedicated input signal phaseTick is provided.

Signal	Rate
phaseTick	480 kHz

The input and output Flow interfaces are synchronized to `phaseTick`. All signals remain synchronous to the 24 MHz main system clock.

Internal: Processing frame

One frame of the external sample sync is 50 main clock cycles long. Calculations can be distributed across multiple system clock cycles during this. Internal processing is not required to use a fully parallel or serial datapath; mixed design is allowed.

Modules

SVF (Top level)

SVF combines `FilterCore`, `FilterMux`, and `ParameterMapper`, and handles the connection of all input and output signals.

Filter Core

`FilterCore` is a Chamberlin State Variable Filter (SVF). It supports runtime adjustment of:

- Cutoff
- Resonance

and it provides these outputs simultaneously:

- Lowpass (`lp`)
- Bandpass (`bp`)
- Highpass (`hp`)

The name SVF (State Variable) refers to two of these passes being internal state variables:

- Lowpass state (`lp`)
- Bandpass state (`bp`)

Calculation

For each input sample, the Highpass (`hp`), Bandpass (`bp`) and Lowpass (`lp`) outputs are calculated from the current filter states, and then the states are updated.

Basic equations:

```
hp = input - lp - resonance * bp
bp = bp + cutoff * hp
lp = lp + cutoff * bp
```

Per sample, the filter requires:

- 3 multiplications
- 4 add/sub operations

The algorithm operates entirely on fixed-point values and uses only:

- Registers
- Adders/Subtractors

- Multipliers

Multiplier results use extended precision internally. After each multiplication, the result shall be rescaled (downshifted) to the internal state width before being used in subsequent calculations or written back into a state register.

The internal state width remains constant throughout the filter pipeline and shall not grow between processing stages.

Example Width Propagation

```
bp(24) * resonance(8)
  -> 32 bit product
  -> downshift by 8
  -> 24 bit result

input(16)
  -> sign extend
  -> 24 bits

input(24) - lp(24)
  -> 25 bit result

(input - lp)(25) - resBp(24)
  -> 26 bit result

resize
  -> hp(24)

hp(24) * cutoff(12)
  -> 36 bit product
  -> downshift by 12
  -> 24 bit result

bp(24) + scaledProduct(24)
  -> 25 bit result

resize
  -> bp(24)

bp(24) * cutoff(12)
  -> 36 bit product
  -> downshift by 12
  -> 24 bit result

lp(24) + scaledProduct(24)
  -> 25 bit result

resize
  -> lp(24)
```

Arithmetic operations may temporarily increase signal widths. Before values are stored into state registers or used as the next state variable, they shall be resized to the defined internal state width.

The state registers `lp` and `bp` remain 24 bits wide throughout operation.

Filter Mux

`FilterMux` is responsible for output selection. The initial implementation shall support selection of:

- Lowpass
- Bandpass
- Highpass

responses.

It is also responsible for downsizing the internal 24-bit representation back to the 16-bit output. To prevent harsh wrap-around distortion when filter outputs exceed 16-bit signed boundaries (due to filter peaking or phase-shift overshoot), the module shall apply a saturating clamp to output values, limiting output samples strictly to `[-32768, 32767]`.

Parameter Mapper

`ParameterMapper` converts user-facing parameters into internal filter coefficients.

Cutoff Mapping

- Input: `UInt(8)`
- Output: `UInt(12)`
- ROM: `256 x 12`
- Mapping: exponential (log-like frequency distribution)

Resonance Mapping

- Input: `UInt(8)`
- Output: `UInt(8)`
- ROM: `256 x 8`
- Mapping: quadratic response curve

Types and Widths

Item	Type
<code>sampleIn</code>	<code>SInt(16 bits)</code>
<code>sampleOut</code>	<code>SInt(16 bits)</code>
<code>lp state</code>	<code>SInt(24 bits)</code>
<code>bp state</code>	<code>SInt(24 bits)</code>
<code>hp signal</code>	<code>SInt(24 bits)</code>
<code>cutoff</code>	<code>UInt(8 bits)</code>
<code>resonance</code>	<code>UInt(8 bits)</code>
<code>cutoffCoeff</code>	<code>UInt(12 bits)</code>
<code>resonanceCoeff</code>	<code>UInt(8 bits)</code>

Control Signals

Signal	Type
enable	Bool
mode	UInt(2 bits)

Mode encoding:

Value	Response
00	Lowpass
01	Bandpass
10	Highpass
11	Reserved

The control signals are inputs to the top-level SVF module and distributed internally.

When `enable` is deasserted, the module output shall be zero.

Bypass functionality is handled outside of the filter module.